



Sistema de Gestão de Estoque em PostgreSQL

Projeto de **Gestão de Estoque** desenvolvido inteiramente em **SQL (PostgreSQL)**, com foco em **boas práticas de modelagem, integridade de dados, uso profissional de triggers, funções e views**, simulando um cenário real de controle de entradas e saídas de produtos.

Este projeto foi pensado como **case de portfólio**, demonstrando domínio de lógica de banco de dados, regras de negócio no nível do banco e organização profissional de scripts SQL.



Objetivo do Projeto

Criar um sistema de estoque capaz de:

- Controlar produtos e locais de estoque
- Registrar movimentações (entrada e saída)
- Atualizar automaticamente o estoque via **trigger**
- Garantir integridade e consistência dos dados
- Disponibilizar **views analíticas**, incluindo visão financeira

Todo o controle crítico ocorre **no banco de dados**, sem depender da aplicação.



Tecnologias Utilizadas

- PostgreSQL
 - SQL / PLpgSQL
 - pgAdmin 4 (ambiente de testes)
-



Estrutura do Projeto

```
sql/  
├─ 01_schema.sql           # Criação das tabelas  
├─ 02_dados_base.sql       # Dados iniciais (opcional)  
├─ 03_triggers_functions.sql # Funções e triggers  
├─ 04_views.sql            # Views analíticas  
README.md
```



Principais Entidades

- **produtos** – Cadastro de produtos
- **locais_estoque** – Locais físicos de armazenamento
- **estoque** – Quantidade atual por produto/local
- **movimentacoes** – Histórico de entradas e saídas

- **usuários** – Responsáveis pelas movimentações
-

Arquitetura de Regras de Negócio

Atualização de Estoque (Ponto-chave do projeto)

A atualização do estoque segue uma **arquitetura limpa e profissional**:

- A **função** `registrar_movimentacao`:
 - Valida os dados
 - Insere o registro na tabela `movimentacoes`
- O **trigger** `trg_atualiza_estoque`:
 - É o **único responsável** por atualizar a tabela `estoque`
 - Decide automaticamente se deve **INSERT** ou **UPDATE**

 Isso evita duplicidade de regras e garante previsibilidade.

Função Principal

`registrar_movimentacao(...)`

Responsável por registrar qualquer entrada ou saída de produto.

Características: - Validação de tipo de movimentação (`ENTRADA` / `SAIDA`) - Registro do usuário responsável - Não altera estoque diretamente (responsabilidade do trigger)

Exemplo de uso:

```
SELECT registrar_movimentacao(  
  1, -- produto_id  
  1, -- local_estoque_id  
  1, -- usuario_id  
  'ENTRADA',  
  10,  
  'Compra inicial'  
);
```



Trigger de Atualização de Estoque

- Executado **AFTER INSERT** em `movimentacoes`
- Regras:
- Se existir estoque → UPDATE

- Se não existir → INSERT
 - Garante que nunca haja movimentação sem reflexo no estoque
-



Views Criadas



View de Estoque Atual

Apresenta: - Produto - Local de estoque - Quantidade atual



View Financeira

Apresenta: - Total de entradas - Total de saídas - Saldo financeiro por produto

Essas views simulam relatórios reais utilizados por áreas administrativas.



Testes

O projeto foi testado com: - Entradas sucessivas - Saídas com validação de saldo - Criação automática de estoque inexistente - Registro de usuário responsável



Como Executar o Projeto

1. Crie um banco no PostgreSQL
2. Execute os scripts na seguinte ordem:

```
01_schema.sql  
02_dados_base.sql (opcional)  
03_triggers_functions.sql  
04_views.sql
```

1. Utilize a função `registrar_movimentacao` para operar o sistema
-



Diferenciais do Projeto

- Regras críticas no banco (não na aplicação)
 - Uso profissional de triggers
 - Tratamento de cenários reais (estoque inexistente)
 - Código organizado e documentado
 - Ideal como **case SQL para portfólio**
-

Observações Finais

Este projeto foi desenvolvido com foco em **qualidade, clareza e boas práticas**, simulando demandas comuns do mercado em sistemas de controle de estoque.

Desenvolvido por **Marcus Vinicius Araujo Barreto**